SUPERTEAM BRASIL · COLOSSEUM

ProofSec

Escrow programável para contratação de segurança ofensiva na Solana

Manual técnico · arquitetura, fluxos, algoritmos e operação



"A IA propõe, a chain impõe."

Versão 0.2 · Outubro 2026

Solana Devnet · Anchor 1.x · AWS Serverless · Next.js 14

Sumário

1. Visão geral 2
2. Arquitetura 3
3. Fluxo ponta a ponta 4
4. Máquina de estados e instruções 5
5. Contas on-chain e liquidação 6
6. Agente de IA e política de gasto 7
7. Evidências, criptografia e login 8
8. Backend: API e indexação 9
9. Manual de execução e testes 10
10. Revisão, riscos e pendências 11

1 Visão geral

Contratar segurança ofensiva tem uma **assimetria de confiança dupla**. O cliente teme pagar adiantado e receber um relatório genérico. O pesquisador teme entregar a prova de conceito e a empresa corrigir a falha sem pagar. Há ainda um problema **legal**: testar sem autorização formal é crime.

O ProofSec resolve isso com um programa na Solana que trava o pagamento, registra a autorização e a entrega, e só libera o dinheiro por regras públicas.

💼 Empresa

Trava USDC e grava on-chain o hash do escopo autorizado. Aprova ou contesta a entrega dentro da janela de revisão.

🔎 Pesquisador

Aceita o job, entrega o relatório cifrado e grava o SHA-256 on-chain. Recebe na aprovação ou, se o cliente se calar, saca sozinho.

🤖 Agente de IA

Transforma alertas (GuardDuty/Security Hub) em propostas. Abre escrows com uma chave sem custódia, limitada pela política on-chain.

Princípios de projeto

Princípio	Como aparece no sistema
A chain é a fonte da verdade	Status, valores e hashes vivem no programa. O DynamoDB é só espelho e metadados, e o frontend lê a chain diretamente.
Ninguém trava o dinheiro	Liberação otimista: sem resposta do cliente até o fim da janela, o pesquisador saca. Contestação vai para um árbitro.
A IA propõe, a chain impõe	A LLM só redige texto. O orçamento sai de uma tabela determinística e o limite de gasto é verificado pelo programa.
Mínimo de confiança no servidor	Relatórios cifrados no navegador, chave da sessão vinda da assinatura da carteira (SIWS) e backend sem acesso às chaves.
Autorização verificável	`scope_hash` : SHA-256 do escopo e das regras de engajamento, datado on-chain.

2 Arquitetura

Três domínios: o **navegador** (assina tudo e cifra relatórios), a **Solana** (dinheiro e regras) e a **AWS** (metadados, arquivos, login e o cérebro do agente). O executor do agente roda fora da AWS, com uma chave de poder limitado.

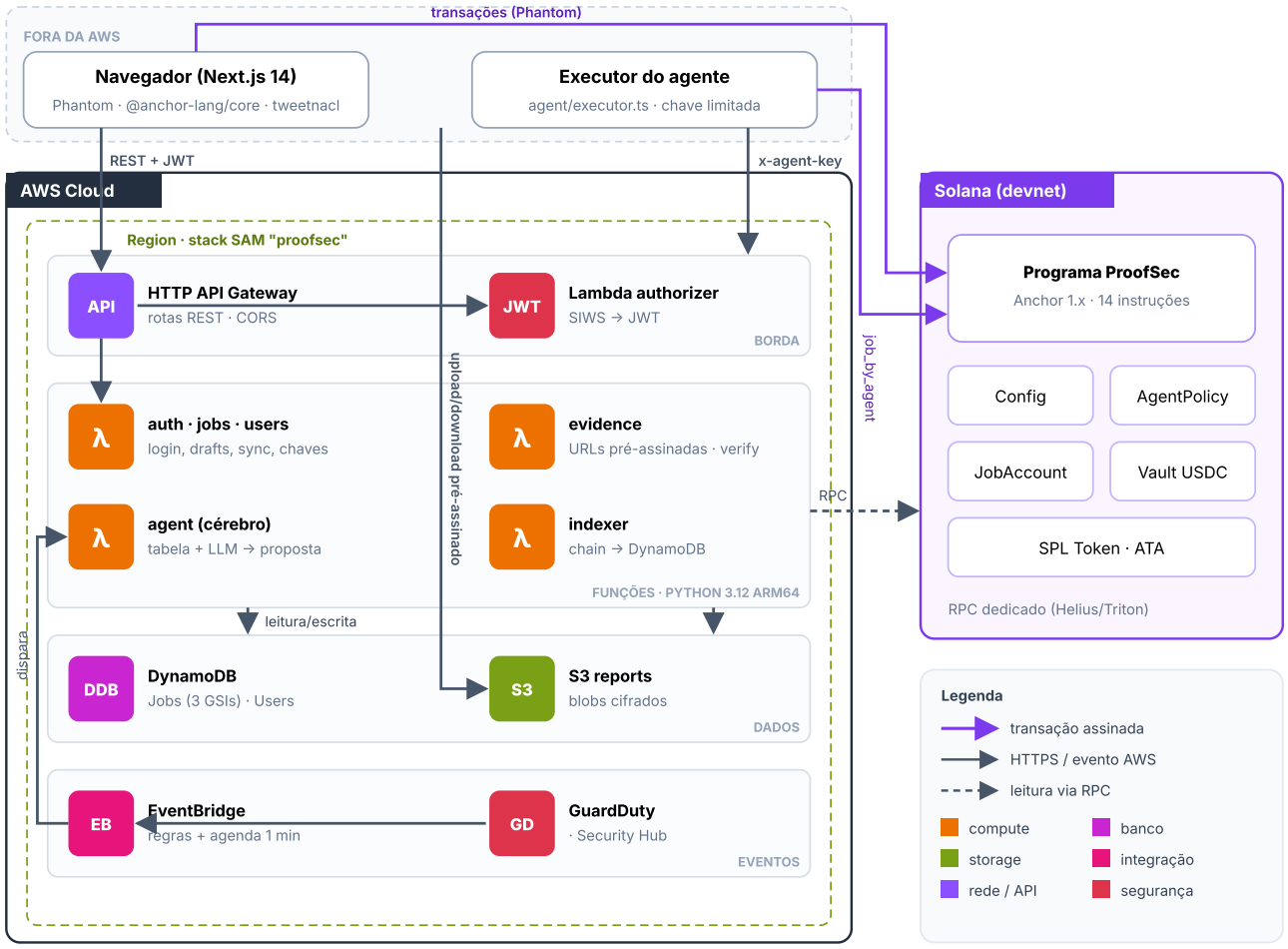


Figura 1 — Arquitetura de referência no estilo AWS. O navegador fala direto com a Solana (transações) e com o S3 (upload/download por URL pré-assinada). O backend nunca segura chaves nem relatórios em claro.

Componente	Responsabilidade	Código
Programa ProofSec	Dinheiro, regras, prazos, limites do agente	`contracts/programs/proofsec/src`
Frontend	Carteira, transações, criptografia, verificação de hash	`frontend/src`
Backend SAM	Login SIWS, metadados, S3, indexador, cérebro do agente	`backend-serverless/`
Executor	Abre escrows aprovados pela política	`agent/executor.ts`

3 Fluxo ponta a ponta

Sequência completa do caminho feliz, com a bifurcação de liquidação no final.

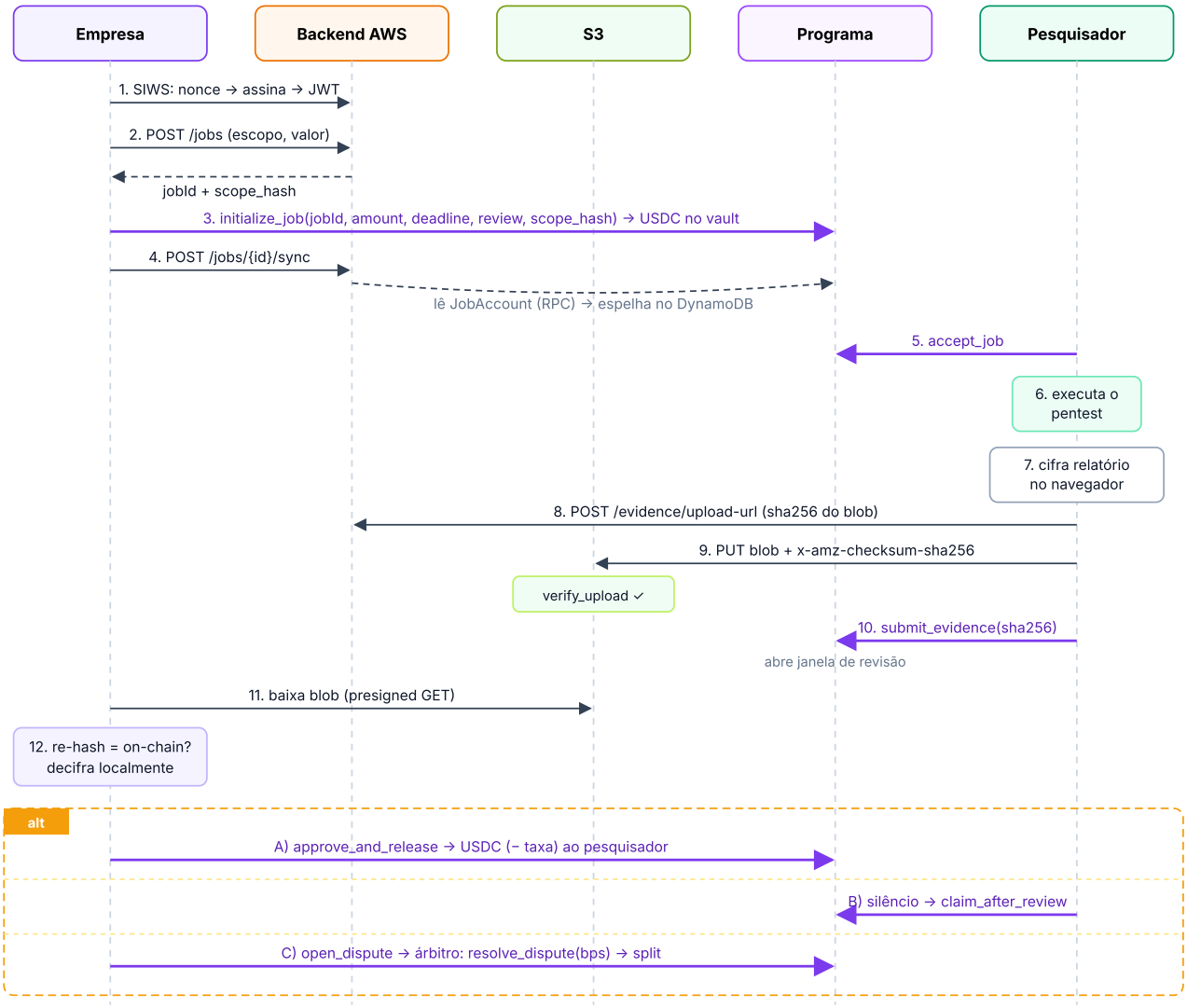


Figura 2 — Diagrama de sequência. Setas roxas são transações on-chain; pretas são HTTPS; tracejadas são leituras.

Modo só on-chain. Sem `NEXT_PUBLIC_API_URL`, os passos 1, 2, 4, 8, 9 e 11 somem. O navegador monta o escopo e calcula o hash sozinho, e o relatório é entregue por fora. Os passos 3, 5, 10 e a liquidação continuam iguais.

4 Máquina de estados e instruções

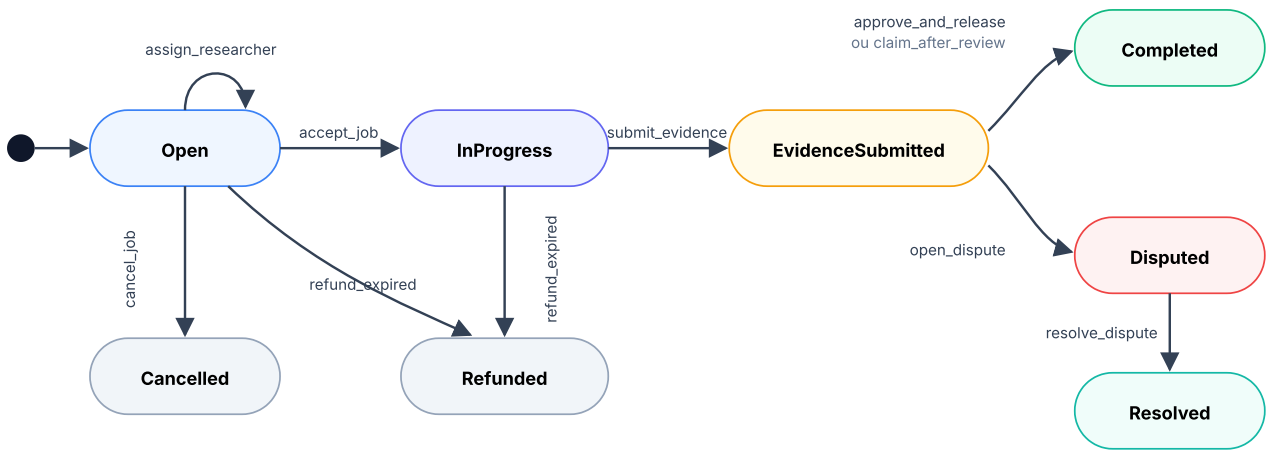


Figura 3 — Estados de `JobAccount.status`. Estados finais: Completed, Resolved, Refunded, Cancelled. Em todo estado final o vault é zerado e fechado.

Instrução	Quem assina	Pré-condição	Efeito
`initialize_config`	admin (1ª vez)	config não existe	árbitro, tesouraria, taxa ≤ 10%, mint permitido
`set_agent_policy`	empresa	max_job ≤ max_período	cria/atualiza limites do agente
`initialize_job`	cliente	prazo ≥ 10 min, revisão 60 s a 30 d, hash ≠ 0, mint ok	USDC → vault; status Open
`initialize_job_by_agent`	agente	política ativa e dentro dos limites; PDA é delegate	idem, debitando a empresa via delegate
`assign_researcher`	cliente	Open	restringe quem pode aceitar
`accept_job`	pesquisador	Open, antes do prazo, ≠ cliente, atribuído ou livre	InProgress
`submit_evidence`	pesquisador vinculado	InProgress, antes do prazo	grava sha256; abre janela de revisão
`approve_and_release`	cliente	EvidenceSubmitted	paga pesquisador (− taxa); fecha vault
`claim_after_review`	pesquisador	EvidenceSubmitted e janela encerrada	idem (liberação automática)
`open_dispute`	cliente	EvidenceSubmitted, janela aberta	Disputed
`resolve_dispute(bps)`	árbitro	Disputed	split pesquisador/cliente; Resolved
`refund_expired`	cliente	Open/InProgress e prazo vencido	devolve tudo; Refunded
`cancel_job`	cliente	Open	devolve tudo; Cancelled

5 Contas on-chain e liquidação

Endereços derivados (PDAs)

Conta	Seeds
Config	`["config"]`
JobAccount	`["job", client, job_id]`
Vault	`["vault", client, job_id]` token account; autoridade = PDA do job
AgentPolicy	`["policy", owner]`

Incluir o `client` nas seeds impede que alguém "ocupe" o `job_id` de outra empresa.

Layout do JobAccount (bytes)

0 discriminator (8)
8 client: Pubkey ← memcmp
40 researcher: Pubkey ← memcmp
72 mint · 104 vault
136 amount u64 · 144 deadline i64
152 review_period · 160 review_deadline
168 scope_hash [u8;32]
200 evidence_sha256 [u8;32]
232 status u8 ← memcmp
233 by_agent · 234 bump · 235 vault_bump
236 job_id: String (u32 len + ≤32)

Campos fixos primeiro → filtros memcmp em offsets estáveis (indexador e frontend sem PDA derivation).

Algoritmo — liquidação (`settle`)

```
// Usado por approve_and_release, claim_after_review e resolve_dispute
function settle(vault, researcher_gross, fee_bps):
    total          ← vault.amount                // saldo REAL (não job.amount): doações ao vault não travam o fechamento
    require researcher_gross ≤ total
    fee            ← ⌊researcher_gross × fee_bps / 10 000⌋     // u128, sem overflow
    researcher_net ← researcher_gross − fee
    client_amount  ← total − researcher_gross
    transfer_checked(vault → ATA(researcher), researcher_net)   // ATA derivada: cliente não consegue desviar
    transfer_checked(vault → ATA(treasury),   fee)
    if client_amount > 0: transfer_checked(vault → ATA(client), client_amount)
    close_account(vault → rent ao cliente)     // assinado pelo PDA do job

// approve / claim: researcher_gross = total
// resolve_dispute(bps): researcher_gross = ⌊total × bps / 10 000⌋
```

Garantias de segurança (correções C1, C2 e A1–A7)

Ataque	Defesa no código
Cliente desvia o pagamento para si	`associated_token::authority = researcher` + `has_one = researcher`
Terceiro drena job expirado	`has_one = client` ; destino é a ATA do cliente
Token falso exibido como USDC	`config.allowed_mint` + `has_one = mint`
Vault de outro job	`has_one = vault` (pubkey gravada no job)
Pesquisador ou cliente trava os fundos	janela de revisão + `claim_after_review` + disputa
Prazo no passado (golpe do refund)	`deadline ≥ now + 10 min`

6 Agente de IA e política de gasto

O agente tem duas partes: o **cérebro** (Lambda, sem chaves) decide o que propor e as **mãos** (executor, chave limitada) assinam. Mesmo que as duas partes sejam comprometidas, o teto continua valendo, porque quem confere os limites é o programa.

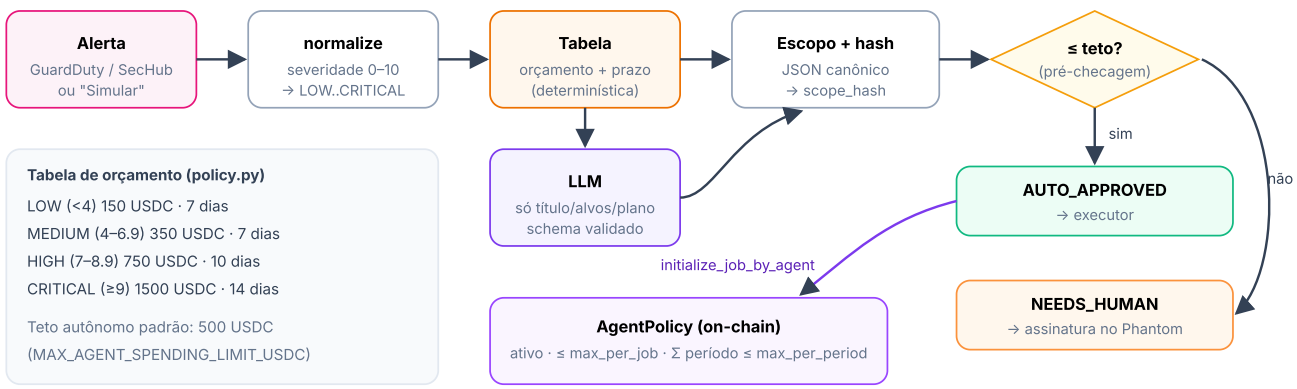


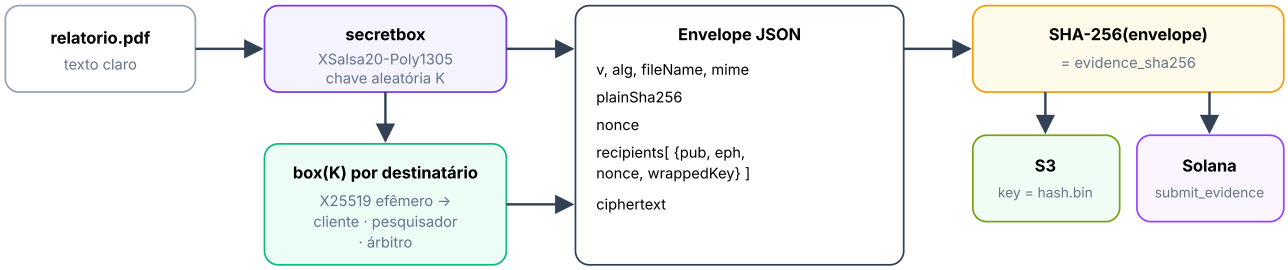
Figura 4 — Pipeline do agente. A LLM nunca toca em valores; a decisão final é do programa.

Algoritmo — verificação on-chain do agente

```
instruction initialize_job_by_agent(job_id, amount, deadline, review, scope_hash):
    validate_new_job(...)                                  // mesmas regras do job humano
    p ← AgentPolicy[owner]; require signer == p.agent ∧ p.active
    require amount ≤ p.max_per_job
    if now ≥ p.period_start + p.period_seconds:             // janela deslizante simples
        p.period_start ← now; p.spent_in_period ← 0
    require p.spent_in_period + amount ≤ p.max_per_period
    p.spent_in_period += amount
    transfer_checked(ATA(owner) → vault, amount, authority = PDA(policy))   // delegate SPL
    job.client ← owner ; job.created_by_agent ← true      // reembolso volta para a empresa
```

Por que delegate SPL? A empresa aprova o PDA da política (`approve_checked`) com uma allowance. Só o programa assina por esse PDA, e só dentro dessa instrução. A chave do agente sozinha não move nada, e o dinheiro movido sempre vai para um escrow reembolsável da própria empresa.

7 Evidências, criptografia e login



Chave de cada carteira (sem expor a seed):

sig ← signMessage("ProofSec encryption key v1 …") ; seed ← SHA-256(sig) ; (pk, sk) ← X25519(seed). Ed25519 é determinístico ⇒ a mesma chave sempre.

Figura 5 — Cifragem para vários destinatários. O backend guarda só a chave pública (PUT /users/me/encryption-key).

Algoritmo — verificação pelo cliente

```
blob ← GET presigned(reports/{jobId}/{sha}.bin)
if SHA-256(blob) ≠ job.evidence_sha256 (on-
chain):
    abort "hash diferente"
kp   ← derive_keypair(signMessage)
r    ← envelope.recipients[pub == kp.pk]
K    ← box.open(r.wrappedKey, r.nonce, r.eph,
kp.sk)
pdf  ← secretbox.open(ciphertext, nonce, K)
require SHA-256(pdf) == envelope.plainSha256
```

Defesa em profundidade no upload: o PUT pré-assinado exige `x-amz-checksum-sha256`, e a Lambda `verify_upload` recalcula o hash e apaga o objeto se ele divergir.

Algoritmo — Sign-In With Solana

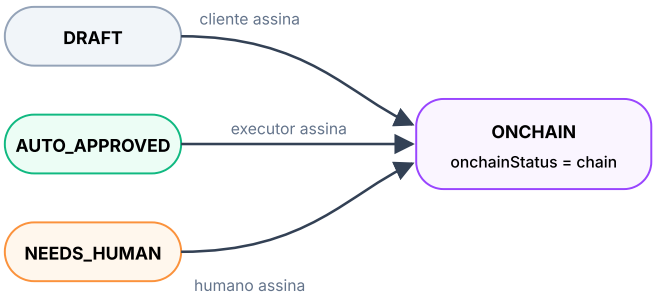
```
POST /auth/nonce {wallet}
   nonce ← random(16) ; msg ←
domínio+wallet+nonce+ts
   Users[wallet].pendingMessage ← msg (TTL 5 min)
signature ← Phantom.signMessage(msg)
POST /auth/verify {wallet, msg, signature}
   require msg == pendingMessage ∧ não expirado
   require Ed25519.verify(wallet, msg, signature)
   apaga nonce (uso único)
   return JWT{sub: wallet, exp: +12h}
Rotas protegidas: Lambda authorizer valida o JWT
e injeta a wallet no contexto (nunca vem do
body).
```

8 Backend: API e indexação

Rota	Auth	Função
`POST /auth/nonce` · `/auth/verify`	—	Login SIWS → JWT
`GET /jobs[?wallet=]`	—	Marketplace: só jobs que já existem on-chain
`GET /jobs/mine`	JWT	Inclui rascunhos e propostas do agente da carteira
`GET /jobs/{id}`	—	Detalhe de job on-chain
`POST /jobs`	JWT	Cria DRAFT; devolve `jobId` e `scopeHash`
`POST /jobs/{id}/sync`	—	Reflete o estado on-chain (idempotente)
`POST /evidence/upload-url`	JWT	Só o pesquisador vinculado on-chain, com o job InProgress
`POST /evidence/download-url`	JWT	Cliente, pesquisador ou árbitro; chave derivada do hash on-chain
`PUT /users/me/encryption-key` · `GET /users/{w}/…`	JWT · —	Chave pública X25519
`POST /agent/propose`	JWT	Alerta simulado → proposta
`GET /agent/proposals` · `POST …/{id}/executed`	x-agent-key	Fila do executor

Funções sem rota: `indexer.handler` (EventBridge, a cada 1 min), `evidence.verify_upload` (evento do S3) e `agent.on_event` (regras do GuardDuty/Security Hub).

Ciclo do item no DynamoDB



Algoritmo — reconciliação (`apply_onchain`)

```
onchain ← decode(JobAccount)
item    ← DynamoDB[onchain.job_id]
if item = ∅: cria registro mínimo
elif item.client ≠ onchain.client
     v item.scopeHash ≠ onchain.scope_hash:
     marca syncError ; não sobrescreve
else: item ← item ⊕ campos on-chain
     status ← ONCHAIN
     evidenceS3Key ← reports/{id}/{sha}.bin
```

A chave do S3 é **derivada do hash on-chain**, nunca do pedido do usuário. Assim o download sempre entrega exatamente o arquivo que foi registrado na chain.

9 Manual de execução e testes

Nível 1 — Backend e agente (2 min)

```
pip install cryptography
npm run test:backend        # 16 testes OK
npm run agent:demo
python agent/agent.py --severity 8.5
```

Nível 2 — Contrato (WSL/Linux)

```
cd contracts && npm install
anchor keys sync
anchor build                # + IDL
anchor test                 # 8 ok + 1 lento
anchor deploy --provider.cluster devnet
```

Nível 3 — Frontend com a devnet

```
cd frontend
copy .env.example .env.local
npm install
npm run dev   # :3000
```

Nível 4 — Backend AWS

```
cd backend-serverless
sam build
sam deploy --guided
# ApiEndpoint → .env.local
```

Roteiro de demonstração (duas contas no Phantom)

1. **A (empresa):** "Inicializar configuração" (só na primeira vez).
2. **A:** criar escrow de 10 USDC com **Revisão = 0.02 h** (~70 s).
3. **B (pesquisador):** "Aceitar job" → "Registrar evidência" com um PDF.
4. **A:** verificar hash → "Aprovar e liberar" (link do Explorer na notificação).
5. **Liberação automática:** repetir 2–3 sem aprovar; depois de ~1 min, B clica em "Sacar pagamento".
6. **Disputa:** A clica em "Contestar"; o árbitro (A, na demo) divide pelo controle deslizante.
7. **Agente:** `npm run keygen` em `agent/` → pubkey no painel de política → "Simular alerta" (sev. 5) → `npm run executor:once`.

Variáveis de ambiente

Variável	Onde	Valor
`NEXT_PUBLIC_PROGRAM_ID`	frontend	endereço após o `anchor deploy`
`NEXT_PUBLIC_RPC_URL`	frontend	RPC dedicado (o público tem rate limit)
`NEXT_PUBLIC_API_URL`	frontend	saída `ApiEndpoint`; vazio = modo só on-chain
`NEXT_PUBLIC_USDC_MINT`	frontend	`4zMMC9…DncDU` (USDC Circle devnet)
JwtSecret · AgentApiKey · CompanyWallet	sam deploy	≥ 32 / ≥ 24 caracteres · carteira da empresa
`API_URL` · `AGENT_API_KEY` · `USDC_MINT`	executor	mesmos valores do backend

> **Atenção:** rode o `npm run dev` dentro de `frontend/`, ou use o atalho `npm run dev` na raiz (agora existe um `package.json` com scripts).

10 Revisão, riscos e pendências

Estado da verificação

Parte	Status	Detalhe
Backend Python	testado	16 testes: SIWS (incluindo replay e assinatura errada), Decimal, decode on-chain, privacidade de rascunhos, evidência só do pesquisador, política do agente.
Agente (cérebro)	executado	Demo offline com o mesmo código da Lambda.
Contrato Anchor	não compilado	Sintaxe validada com rustfmt e ordem das contas conferida contra o IDL. Falta `anchor build`, porque os registries estavam bloqueados no ambiente de revisão.
Frontend	não buildado	Lógica própria com checagem de tipos OK (bibliotecas externas como stubs). Falta `npm run build`.
IDL	provisório	Gerado à mão (discriminadores calculados). O `anchor build` substitui pelo real.

Achados desta revisão (corrigidos)

- **Privacidade:** `GET /jobs?wallet=` expunha rascunhos e propostas do agente, que revelam o que está vulnerável. Agora a rota pública só lista jobs on-chain, e `GET /jobs/mine` (autenticada) mostra o resto.
- **Usabilidade:** `npm run dev` na raiz falhava. Foi criado um `package.json` de atalhos (`dev`, `test:backend`, `agent:demo`...).
- O frontend recarrega os dados ao fazer login, para trazer as propostas privadas.

Riscos e pontos de atenção

Risco	Mitigação / próximo passo
API do Anchor 1.x (CPI, contas)	Primeiro `anchor build`; erros prováveis são pontuais (assinaturas de CPI e nomes de feature).
`initialize_config`: quem chamar primeiro vira admin	Inicializar logo após o deploy; V2: restringir à upgrade authority.
Disputa sem prazo para o árbitro	V2: split automático depois de N dias; árbitro como multisig Squads.
RPC público instável em demo ao vivo	Usar Helius/Triton em `NEXT_PUBLIC_RPC_URL` e `SolanaRpcUrl`.
`getProgramAccounts` no indexador	Bom para o MVP; produção: webhook da Helius com os eventos do programa.
Cliente não ativou a criptografia	O upload falha com mensagem clara; na demo, os dois clicam em "Ativar criptografia" antes.

Roadmap V2

Reputação

Atestação não transferível (Solana Attestation Service) emitida no release.

Rendimento

Vault em USDC+ (Reflect): o rendimento durante o pentest vira bônus.

Bug bounty

Marcos por severidade e vários pesquisadores por programa.